

PEARL NACRE GPU

A GPU mining algorithm for LLM inference

Technical design, reference experiments,
and an implementation plan for cuPOW

01 Fuse GPU preparation

Use sparse noise indices and shared-memory correction.

02 Preserve cuPOW

Feed the existing tensor-core work and proof pipeline.

03 Protect LLM serving

Keep numerical profiles fixed and mining within a budget.

01 / DESIGN BRIEF

The recommendation

Build Nacre GPU around fused sparse preparation for Pearl's cuPOW matrix operations. The proposed CUDA kernel forms noisy LLM inputs and their correction factors in one pass per operand row, using sparse indices and shared-memory integer accumulation. A serving controller budgets mining work, and a receipt layer records the strength of service verification.

The deliverable includes actual CUDA C++ prototype source, a GPU self-test, a preparation microbenchmark, and CPU mathematical checks. The kernel targets a concrete preprocessing cost; it preserves the existing tensor-core mining and proof construction. CUDA compilation and GPU execution remain required.

Design choice	What it achieves
Fused GPU preparation	Replaces sparse-factor dense arithmetic in preprocessing with direct index operations and signed shared-memory accumulation.
Budget-aware kernel selection	Uses measured latency, memory headroom, and proof backlog to admit mining work.
Fixed arithmetic profile	Mining decisions do not silently change the model's quantization or sampling behavior.
Typed service receipts	Distinguishes an operator attestation, a sampled audit, and a full execution proof.
Fee-funded service rewards	Clients fund service payments. Self-generated jobs do not receive a new inflation multiplier.
Existing consensus rules	Retains Pearl's proof format, accepted certificate version, rank rules, issuance, and chain selection.

What is new in this proposal

A candidate GPU preprocessing kernel, plus latency reservation and service accounting. Pearl already implements noisy GEMM, fused correction, and static shape selection. Speedup, patent novelty, and production compatibility are not claimed; these require measurement and review.

How to read this report

GPU design and CUDA mapping: pp. 5-8. Serving controller: pp. 9-11. Receipts and verification: pp. 12-15. Runtime and experiments: pp. 16-18. Engineering plan: pp. 19-23. Methods and sources: pp. 24-27.

02 / SOURCE AUDIT

What Pearl already implements

Public repository inspected at commit 2f8b770cac8f (4 October 2026); sources checked 7 October 2026.

Observed implementation	Implication for Nacre
vLLM plugin and noisy GEMM path [3,4]	Integrate at the existing kernel decision point. Do not describe LLM mining as an entirely new Pearl capability.
Shape gate: $m \geq 1024$, $n \geq 256$, $k \geq 1024$; unit dimensions excluded [5]	Retain legality/shape checks, then add measured service and resource gates. These are checked-in defaults, not universal optimal thresholds.
Mining mode quantizes activations to INT7; non-mining mode uses INT8 [4]	Choose noisy or vanilla GEMM inside the INT7 path. Do not toggle the outer mining_enabled setting per request.
Current proof enum includes Int7xInt7ToInt32 [6]	An FP8/FP4 mining design is a separate cryptographic and accuracy project.
Height-dependent certificate, dense-only, rank-penalty, and salted-seed rules [7,8]	Resolve rules from the node at the target height. MoE structures in the code do not mean MoE certificates are eligible on every network.
Asynchronous proof handling already exists [9]	Add bounded queues and admission control; do not claim asynchronous proving as a new feature.

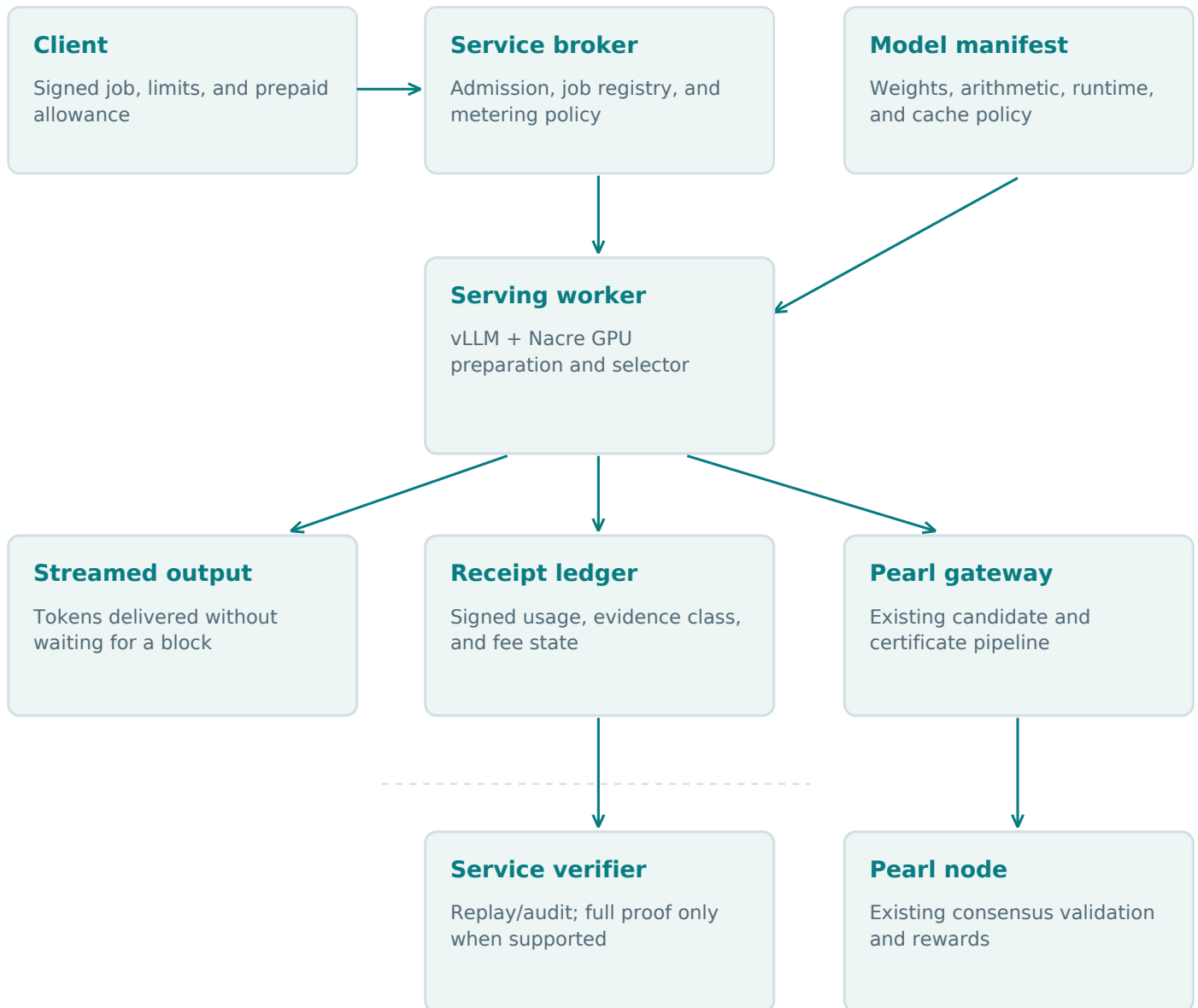
The inspected mainnet parameters set dense-only activation at height 91,630, rank-penalty activation at 96,251, and salted-seed activation at 99,000. This is a code snapshot, not a live-chain status query. Preserve its version-aware logic rather than copying an older seed formula. [7,8]

Pearl and Together have also announced an inference product using Pearl. That establishes a stated deployment direction; it does not independently establish performance for Nacre. Pearl's own research agenda still lists low-precision security and efficient full proof-of-inference as open problems. [2,11]

03 / ARCHITECTURE

Two paths, distinct evidence

Inference delivery stays on the serving path. Mining proofs and service audits run under separate resource budgets.



OFF-CHAIN SERVICE TRUST

A broker or operator attests to the job-to-work link in v0.2. A cuPOW certificate alone does not establish that a particular customer received the requested model output.

CONSENSUS TRUST

Nodes check Pearl work under existing rules. Service receipt scores never alter block difficulty or chain weight.

GPU CORE / 1 OF 4

The GPU algorithm: fused sparse prep

A proposed CUDA implementation of existing noising algebra, followed by Pearl's unchanged tensor-core mining path.

Pearl's noise generator produces sparse factors with one +1 and one -1 per K row. Its noising kernels use tensor-core matrix operations to form noisy operands and correction factors. Nacre GPU proposes a different GPU mapping for that preparation: direct gathers plus signed shared-memory accumulation. It does not replace the hard matrix multiplication or weaken the proof predicate. [15,16]

```
A: M x K      B: N x K (stored transposed weights)
LA: M x R     LB: N x R
SA, SB: K x R, each row has one +1 and one -1

AP = A + LA * transpose(SA)
BP = B + LB * transpose(SB)
U  = A * SB
V  = BP * SA

C = AP * transpose(BP)
  - LA * transpose(V) - U * transpose(LB)
  = A * transpose(B)
```

Represent each sparse row by positive/negative index pairs (pA, nA) and (pB, nB). For one A element, compute **AP[i,k] = A[i,k] + LA[i,pA[k]] - LA[i,nA[k]]**. Add A[i,k] to U[i,pB[k]] and subtract it from U[i,nB[k]]. For the B lane, compute BP with LB and the B indices, then accumulate **BP** into V using the A indices.

The cross term is handled correctly

V uses the noisy B operand. Substituting the original B would omit the noise cross term and produce incorrect LLM output. The recovery equation above follows directly by expanding AP and BP; the CPU oracle checks it against dense arithmetic.

Dense preparation expressions perform $O(RK(M+N))$ arithmetic; the sparse row method performs $O(K(M+N))$ index operations and updates after map construction. This is an operation-count reduction, not an R-fold speedup claim. Tensor cores may still beat shared-memory atomics on particular shapes.

GPU mapping and memory contract

Stage	GPU work and ownership
Canonical map adapter	Read the original SA/SB factors; require exactly one +1 and one -1 per row; pack pA, nA, pB, nB into one uint32 per K. Preserve the official seed generator.
A preparation kernel	One 256-thread block owns one A row. Cache LA in shared memory; form AP with coalesced loads/stores; accumulate U with shared int32 atomics.
B preparation kernel	One block owns one stored B row. Cache LB; form BP; accumulate V using the newly formed BP values and A's sparse indices.
Existing Pearl pipeline	Apply the existing correction conversion/scaling, noisy main GEMM, transcript extraction, hash predicate, and certificate pipeline. This integration is proposed, not wired into the prototype.

What the GPU stores

The prototype allocates AP ($M \times K$ INT8), BP ($N \times K$ INT8), U ($M \times R$ INT32), V ($N \times R$ INT32), and packed maps (4K bytes). Each block uses 128 INT32 bins plus 128 INT8 noise factors: **640 bytes of declared shared arrays**, excluding compiler/runtime overhead. Input and output buffers must not alias.

The index representation replaces two logical $K \times R$ INT8 sparse-factor arrays with 4K bytes. The simple prototype adapter still reads the dense factors, so it does not yet remove their allocation. Emitting the exact same packed indices directly from the existing generator is a later integration optimization, gated by seed-by-seed equality tests.

The performance tradeoff

The fused row pass avoids a second global read for its correction accumulation, but pays two shared atomic updates per input value, index loads, and possible bank contention. A small number of rows can underfill the GPU, and very long rows can create a tail. Keep Pearl's existing tensor-core preparation as the fallback and select a path only from measured per-shape results.

Do not optimize away the work transcript

The main GEMM accumulation order, hashed intermediate states, and target predicate remain the current Pearl implementation. A faster preparation stage is compatible only after its outputs and resulting certificates match the existing pipeline.

GPU CORE / 3 OF 4

CUDA kernel contract

Full source: [nacre_gpu.cu](#), embedded in the PDF and supplied in the accompanying source ZIP.

```
GPU block owns one operand row:
shared factors[R] = row of LA or LB
shared sums[R] = 0
synchronize_block()

for k assigned to this thread:
  ap, an, bp, bn = unpack(canonical_map[k])
  x = input[row,k]
  if A_lane:
    y = x + factors[ap] - factors[an]
    atomic_add(sums[bp], x)
    atomic_add(sums[bn], -x)
  else:
    y = x + factors[bp] - factors[bn]
    atomic_add(sums[ap], y)
    atomic_add(sums[an], -y)
  noisy_output[row,k] = int8(y)

synchronize_block()
correction[row,:] = sums[:]
```

Every thread participates in the two block barriers. Each output row has exactly one owner, so correction results need no global atomics or cross-block reduction. Shared atomic integer addition is order-independent when overflow is excluded. These synchronization primitives are standard CUDA mechanisms. [18]

Prototype precondition	Enforcement / implication
R = 64 or 128; positive sizes	Host launch checks; other ranks require an explicit kernel revision.
A/B in [-64,63]; LA/LB in [-32,31]	Device guards. Valid noisy values lie in [-127,126], inside INT8.
K <= 2^20	Each correction bin has absolute intermediate sum <= 127K < 2^31. Main GEMM and converter limits remain separately enforced.
No error flag after completion	Any invalid index, factor structure, or range invalidates all outputs. Read/resolve the flag before downstream mining uses them.

GPU CORE / 4 OF 4

GPU validation and break-even test

CPU mathematical validation: 100 cases passed. The independent dense oracle checks both noisy operands, both correction matrices, full integer output recovery, and INT8 ranges. Cases span $R = 64/128$, $K = 1/31/64/257/1024$, random indices, and a concentrated index pattern that would stress shared atomic contention. This validates the mathematical transformation, not CUDA synchronization or machine performance.

Actual CUDA source is included, but has not been compiled or executed here. The source contains a GPU/CPU self-test and a CUDA-event preparation microbenchmark. No CUDA compiler or GPU runtime was available in this environment. A successful standalone test still does not prove compatibility with Pearl's kernels or certificate verifier.

```
nvcc -O3 -std=c++17 -arch=native nacre_gpu.cu -o nacre_gpu
./nacre_gpu
./nacre_gpu --bench 1024 4096 4096 128 100
```

Use a CUDA toolkit that supports the installed GPU and `-arch=native`. Windows uses the corresponding executable invocation. The benchmark times both preparation kernels on synthetic hot buffers; it excludes map generation, conversion, main GEMM, hashing, proof creation, and network work. It reports milliseconds, not mining hashrate.

The adoption criterion

```
T_new = map_cost + fused_A + fused_B + conversion
T_old = existing_preparation_for_the_same_shape
Enable Nacre GPU only if T_new < T_old
AND output/certificate equivalence tests pass
AND end-to-end serving quality and SLO gates pass.
```

Measure cold and reused map contexts, real cryptographic index distributions, adversarially concentrated indices, power, occupancy, shared atomic contention, and tail latency. Reproduce the existing FP16 correction scaling exactly, including rounding behavior. The integer identity alone does not guarantee bitwise equality with the shipped floating-point epilogue. [17]

RTX 5090 relevance

The prototype uses ordinary CUDA blocks and shared integer atomics, without Hopper-only warp-group instructions. It is a plausible porting candidate for a 5090, not tested support. The full Pearl tensor-core and proof stack still needs its own Blackwell compatibility validation.

04 / ALGORITHM

The Nacre selection rule

Objective: earn eligible mining work while preserving the service budget of the most constrained request in a batch.

For batch **b**, let **T0** be predicted fixed-profile inference time without mining. For every affected request **r**, compute slack **s_r** from its next deadline after subtracting predicted inference, remaining queueing, and known shared-device work. Use the earliest required first-token or next-token deadline. Long-prefill requests need a remaining-prefill estimate, not only the next kernel time.

```
B_b = min(alpha * T0, max(0, min_r(s_r) - guard))
U_i = conservative incremental latency reserve for op i
W_i = estimated eligible work under current Pearl rules
Select x_i in {0,1} to favor sum_i(x_i * W_i)
subject to sum_i(x_i * U_i) <= B_b
        memory, proof, template, and policy gates
```

Use a cached greedy admission order by **W_i / U_i** for known shapes. It approximates a small knapsack problem; it is not an optimality claim. Unknown or changing shapes default to plain inference until profiled. A nonzero reserve floor prevents division by zero and accounts for launch and accounting overhead.

Hard admission gates

Gate	Required behavior
Model and kernel	Pinned arithmetic profile; supported shape and certificate rules; no unapproved quantization switch.
Service and resources	Positive budget; memory reserve available; no proof high-water mark; no thermal or error circuit breaker.
Work context	Fresh node template; expected proof cost bounded; optional operator profitability threshold satisfied.

What the bound actually guarantees

The algorithm reserves at most **B_b**. Actual extra latency is bounded by **B_b** only if the latency estimates remain valid and include interference. Empirical percentile estimates are not deterministic bounds. Observe overruns, shrink the budget, and disable mining when calibration fails.

05 / REFERENCE LOGIC

Miner pseudocode and worked example

```

on_batch(batch, node_context):
    profile = require_pinned_numeric_profile(batch)
    budget = service_budget(batch)
    candidates = legal_profiled_ops(batch, node_context)
    permits = plan_by_work_per_latency(candidates, budget)

    for op in execution_order(batch):
        A, B, scales = fixed_profile_operands(op, profile)
        if permit_valid(op, permits, node_context) and
            reserve_resources(op) and health_gates_pass():
            C, possible_hit = existing_cupow_gemm(A, B, scales)
            enqueue_hit_or_discard_if_full(possible_hit)
        else:
            C = existing_plain_gemm(A, B, scales)
            continue_model(C)
            record_actual_cost_and_update_guards(op)

    stream_tokens_and_record_service_receipt(batch)
    # Proof creation/submission occurs off this critical path.

```

Execution order and tensor dependencies stay unchanged; the plan selects operations rather than reordering model layers. Recheck gates at launch. If a previous operation overruns, cancel remaining mining permits and continue inference. An in-flight GPU kernel is not assumed to be preemptible.

An illustrative 20 ms serving batch

With $\alpha = 5\%$, 1.5 ms minimum request slack, and a 0.25 ms guard, the mining budget is $\min(1.0, 1.25) = \mathbf{1.0\ ms}$. Three operations reserve 0.45, 0.35, and 0.40 ms and offer 9, 6, and 4 work units. The greedy rule admits the first two: 0.80 ms reserved and 15 units. The third runs normally.

Template changes and fallback

A stale template cancels future mining permits. Already computed inference output can still be delivered if correct. Stale mining candidates are discarded or handled by the existing node policy. Plain inference fallback must use the same INT7 operands and scales.

06 / NUMERICAL CONTRACT

Preserve the model while mining

The model manifest pins weight digests, tokenizer revision, quantization scales and transforms, graph revision, runtime/kernel build, allowed shapes, sampling algorithm and seed, and the KV-cache policy. Changing any of these creates a new profile. Requests name the exact profile they authorize.

Workload	Proposed treatment
Large prefill GEMMs	Primary pilot candidate. Measure incremental noisy-GEMM cost at realistic batch sizes; admit only with first-token slack.
Latency-sensitive decode	Usually a tighter budget. Keep plain execution when shapes are small or next-token slack is insufficient. Do not enlarge batches solely to mine.
Prefix cache hits	Retain useful cache reuse. Count reused input separately and do not claim that cached computation was executed again.
Speculative decoding	Bill delivered target-model tokens under the contract; record draft and rejected work separately. Include all relevant work in resource measurements.
MoE or unsupported operators	Start with dense models. Maintain correct ordinary execution for unsupported work; obey the node's certificate eligibility rules.

vLLM documents chunked prefill and decode-prioritized scheduling as ways to balance latency and throughput. Nacre should consume those scheduling decisions, not defeat them by filling the GPU with extra mining batches. Tune the integration against a pinned vLLM release. [10]

Three different quality checks

Kernel correctness: compare integer products and recovered results for the same operands, including padding, scale application, overflow boundaries, and bias behavior. **Numerical equivalence:** compare complete pinned-profile inference with mining on and off. **Product quality:** evaluate that profile against the best approved non-mining model configuration. Passing the first check does not establish the third.

No untested low-precision shortcut

FP8, BF16, and FP4 may improve ordinary serving economics, but are not drop-in replacements for the inspected integer proof relation. A faster approved serving configuration can make a mining profile unattractive; the benchmark must allow that outcome. [6]

07 / DATA CONTRACT

Jobs and service receipts

A receipt is an accounting object with a declared verification scope. It is not automatically a proof of inference. Use separate service signing keys, a versioned schema, fixed-width integer amounts, and a single atomic ledger authority for the v0.2 pilot.

Object	Required fields
Job manifest	Chain ID; payer key; 32-byte random nonce; model-manifest hash; salted input commitment; generation-parameter hash; deadline; token limits; maximum fee; verifier-profile hash.
Service policy	Input/cache/output rates; cancellation rules; audit retention; dispute process; maximum unsettled streaming window; clock policy. Commit its digest in the verifier profile.
Segment receipt	Job ID; provider ID; sequence; disjoint output token interval; new/cached input counts; output commitment; charge; previous receipt hash; verification class; audit coverage.
Mining observation	Template/certificate context; mining configuration; operation/shape identity; candidate reference if any; timing and resource record. The provider attests to its association with the job.

```
job_id = SHA256("pearl/nacre/job/v1" || 0x00 || E(fields))
E(fields) = concat(u32_le(len(field)) || field)
nullifier = (job_id, segment_sequence)
receipt_id = H_domain(encoded_receipt_body)
```

For the reference subset, fields are ordered as in the job-manifest row, hashes and keys are 32 bytes, and integer fields are unsigned 64-bit little-endian values. Production decoders must reject unknown schema versions, wrong lengths, duplicate fields, and out-of-range values. The CPU harness tests this subset, not a complete network wire implementation.

Store the next sequence and token boundary transactionally before acknowledging settlement. A persistent uniqueness constraint rejects replays across crashes. A mining proof may reference several batched requests, so allocate observed compute once across the batch; do not duplicate the full work count in every customer receipt.

08 / INCENTIVES AND SETTLEMENT

Pay for service without wash subsidies

Keep two ledgers conceptually separate: **consensus rewards** follow Pearl's existing valid-work rules; **service payments** come from the customer's prepaid allowance. Nacre introduces no extra issuance for declaring a job useful, submitting more receipts, or obtaining a high score from an LLM judge.

```
reserved_customer_funds
  = remaining_allowance + provider_payment
  + verifier_payment + service_fee + refund

additional_protocol_issuance_for_service_receipts = 0
```

For example, a 100-unit service charge can allocate 85 to the provider, 10 to auditing, and 5 to the broker. Those percentages are illustrative, not a recommended token schedule. If one entity is both client and provider, it recycles its own money; a receipt does not mint an additional reward. If it also controls the auditor and broker, fees can recycle too. Cryptography still cannot certify independent economic demand.

Segment state and payment finality

Transition	Condition and failure handling
Reserved -> running	Broker confirms funded allowance and signed model/service terms. Bound work that can run before the next funding check.
Running -> delivered	Stream output off-chain. Record metering evidence. Client acknowledgments help but do not alone solve a dishonest-client dispute.
Delivered -> payable	Apply agreed verification and dispute window. A timeout is an accounting rule, not cryptographic proof of delivery.
Payable -> settled	Commit the unique receipt and payment atomically. Reconcile transfers by idempotent payment ID and handle reorgs before final payout credit.
Cancel or expire	Stop at a safe serving boundary; charge only contractually payable segments; release the unused reservation once.

The first pilot uses a disclosed broker ledger and ordinary payments. Native trustless escrow, automatic slashing, or a new on-chain receipt interpreter requires a separate protocol design and review; this proposal does not assume an EVM execution layer exists. Subsidy-funded discounts also carry price, difficulty, maturity, and orphan risk. [1,7]

09 / VERIFICATION

State exactly what is verified

Evidence class	What it establishes	Trust and limitation
V0: service attested	Provider/broker signs the job, usage, and output record.	Trusts the operator and broker. A hash binds data; it does not prove execution.
V1: sampled audit	Selected committed operations or requests pass the agreed audit.	Report sampled units and coverage. Unsampling execution remains unchecked.
V2: full execution proof	A proof verifies the pinned model computation and output relation.	Requires a specified proof system covering the entire graph. Future research path; not implemented by Nacre v0.2.
P: Pearl work proof	A certificate passes existing cuPOW and node checks.	Separate from V0-V2. It does not by itself prove a complete customer job.

An optional exact matrix audit

For committed A ($m \times k$), B ($k \times n$), and claimed C ($m \times n$), a verifier can draw a fresh secret uniform vector r over a finite field and check $A(Br) = Cr$. For a fixed incorrect matrix relation over that field, one round has false-accept probability at most $1/p$; independent rounds reduce it to $p^{-(t)}$. [12]

The verifier must have the necessary authenticated data or a sound proof of each required access. A few Merkle openings are not enough to evaluate an arbitrary whole-matrix check. Choose the challenge only after commitments and data availability are fixed. Reused, predictable, or grindable challenges invalidate the simple bound.

For a proposed $p = 2^{61} - 1$ field, three rounds give a conditional algebraic bound below 2^{-182} . Exact arithmetic must prevent host integer overflow and modular aliasing: bound both candidate and true integer outputs to a range narrower than p , or prove an equivalent safe encoding. This is not an end-to-end system security claim.

A matrix check is not a transformer proof

Normalization, attention/softmax, positional transforms, routing, KV-cache transitions, and sampling also affect outputs. A valid proof of the wrong model, or a correctly executed low-quality model, does not satisfy the intended service. zkLLM illustrates specialized full-model proof research, not a free upgrade to Pearl. [13]

10 / ADVERSARIAL REVIEW

Threats and residual risks

Attack or failure	Nacre response	What remains
Self-paid fake jobs	No useful-work issuance multiplier; funded fee conservation.	Independent demand is not cryptographically provable. Usage can still be inflated.
Cheaper model or fabricated tokens	Pin model manifest; audits and explicit verification class.	V0 depends on trust; V1 can miss selective cheating.
Replay / double charging	Job nonce, domain separation, receipt sequence, disjoint ranges, atomic uniqueness.	Requires durable storage and idempotent recovery; a Python model is insufficient.
Challenge grinding / collusion	Private verifier randomness after binding data; commit before selection; track aborts.	A colluding verifier can lie. Public randomness needs a bias-resistant protocol.
Withheld audit data	Pre-agreed retention, bounded retrieval time, and contract-defined failure handling.	Privacy and bandwidth costs may make broad audits uneconomic.
Late kernels / proof overload	Conservative reserves, queue caps, memory reservations, mining circuit breaker.	In-flight kernels and shared hardware can still miss deadlines.
Stale proof / cached commitments	Versioned template handles and context-keyed caches.	Network delay still creates stale work. Do not reuse header-keyed roots blindly.
Prompt or activation leakage	Minimize logs; salted commitments; encrypted transfer; approved verifier access.	A normal inference operator sees plaintext. A network ZK proof does not hide inputs from it.
Broker or pool concentration	Portable receipts, multiple brokers, published fees and payout policy.	No assumed Sybil-resistant committee or guaranteed decentralization.

A bond can deter a bounded cheating gain only when detection and adjudication are credible: expected detected loss must exceed the gain. Large bonds do not fix collusion, cheap identities, ambiguous floating-point discrepancies, or unobservable service delivery. v0.2 uses contractual disputes rather than automatic consensus slashing.

11 / EXECUTION CONTROLS

Bound the costs outside the kernel

Proof queue and resource ownership

Reserve bounded memory for each admitted mining launch and its maximum retained candidate data. Count GPU buffers, host buffers, transfers, proof workers, and event callbacks. Use a high-water mark to stop new mining and a lower restart threshold to avoid oscillation. A full queue drops mining opportunities according to a documented policy while continuing inference.

Budget proof-generation CPU and any GPU work explicitly. A background thread can still contend for memory bandwidth, CPU scheduling, and device time. Isolate proving resources where possible; include residual contention in the serving predictor. When a worker fails, release reservations exactly once and preserve the model stream if possible.

Template, seed, and cache handling

A cache key includes model profile, shape, device, arithmetic/kernel version, mining configuration, certificate/seed version, and relevant template context. Cache safe layouts and preprocessing. Recompute header-keyed commitments and context-dependent noise through Pearl's current implementation whenever that context changes. Do not replace salted V3 seed logic with older formulas. [6,8]

Privacy and data availability

Commit private inputs with a fresh 32-byte salt and send the opening only to authorized parties. A salt helps prevent guessing short prompts from a public commitment; it does not encrypt a prompt. Keep prompts, activations, and raw model traces out of public receipts. Encrypt transport and retained audit material, enforce retention limits, and record whether the customer permitted independent audit access.

Fallback is part of the protocol

When paid demand disappears, optional ordinary mining can continue as its own lane. It remains valid work under existing consensus but is not counted as customer inference. When paid traffic arrives, stop admitting fallback work and account for any non-preemptible kernel still in flight. This preserves the distinction between chain liveness and useful service.

Record every fallback reason: unsupported shape, deadline pressure, queue pressure, stale template, unknown profile, thermal limit, or numerical failure. These counts identify whether more engineering effort will actually improve useful LLM throughput.

12 / SYNTHETIC RESULTS

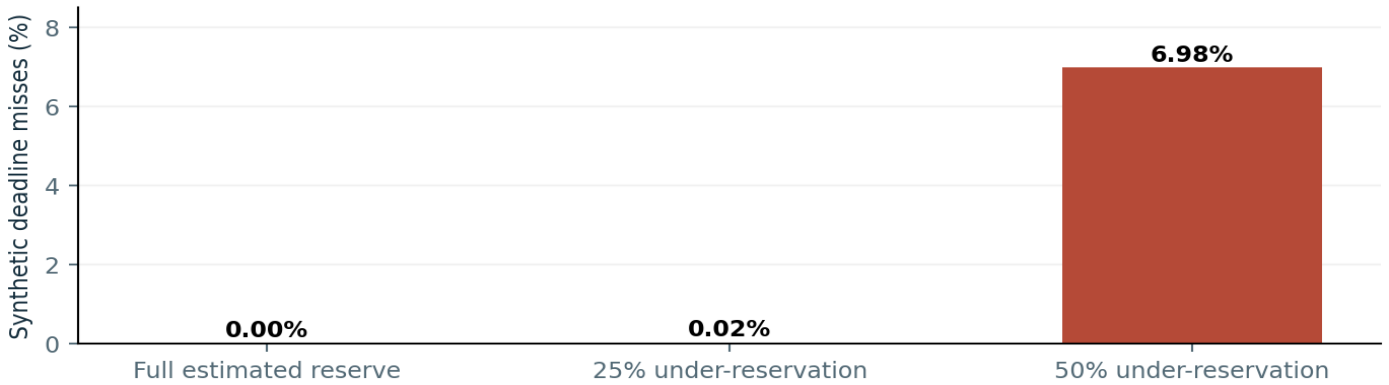
Reference experiment: scheduling

CPU simulation, seed 20261007. These values are mechanism illustrations, not Pearl or GPU performance measurements.

Policy	Work kept	P99 added time	Deadline misses	Capacity proxy
Plain fixed profile	0.0%	0.00%	0.00%	100.00%
Shape-only proxy	100.0%	14.37%	29.72%	95.93%
Nacre: 2% budget	45.9%	1.91%	0.00%	98.84%
Nacre: 5% budget	67.1%	4.41%	0.00%	97.68%
Nacre: 10% budget	71.2%	7.95%	0.00%	97.37%

The 5% policy retains **67.1%** of the trace's eligible work units, with **4.41%** P99 added time and zero observed deadline misses in this trace. Its median added time is 0.594 ms. The shape-only row is a simplified comparator, not a benchmark of the shipped Pearl miner.

There are 10,000 independent batches, 12 candidate operations per batch, random deadline slack, and rare latency shocks. Work units are synthetic weights, not PRL revenue or validated hashrate. The capacity proxy is total baseline time divided by total time after added mining; it is not an arrival-queue or token-throughput model.



Negative result: at 50% under-reservation, the same 5% policy misses **6.98%** of deadlines. Even with full estimated reserves, 35 of 10,000 batches exceed the selected overhead budget due to shocks; the deadline guard absorbs them in this trace. Zero observed misses is not a production guarantee.

Engineering conclusion: calibration, realistic interference estimates, and a circuit breaker are essential. An aggressive estimate can turn higher mining yield into worse service.

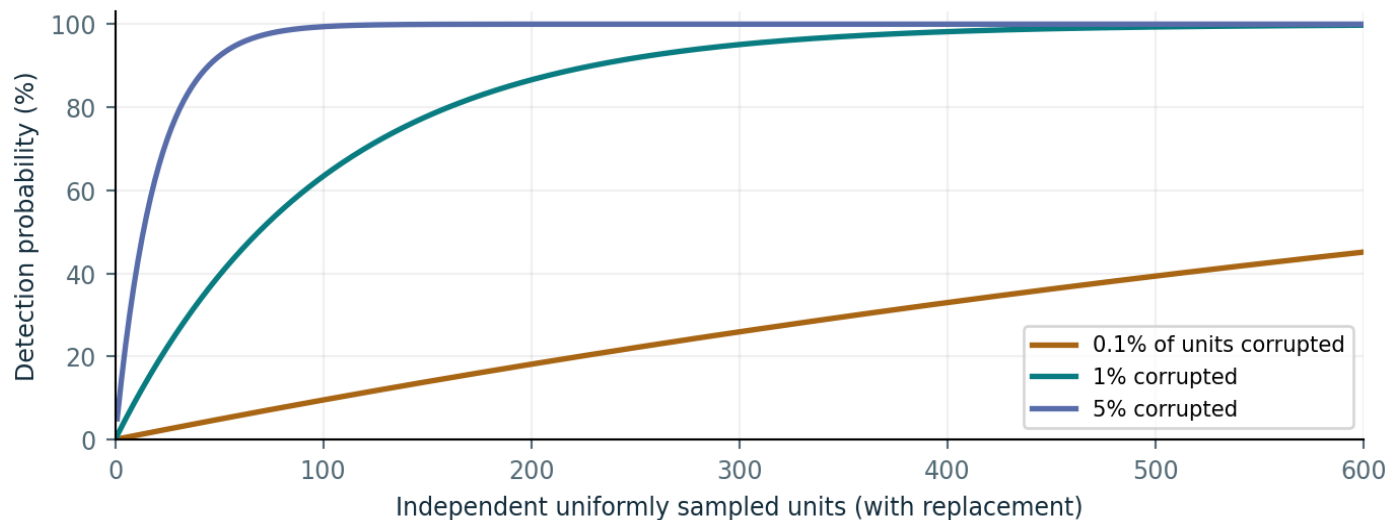
13 / CPU MECHANISM CHECKS

Reference experiment: evidence limits

15 reference checks passed. They cover job-field binding, encoding separation, receipt replay and overlap, incorrect job identity, negative fees, token and spending caps, authenticated-message tampering, fund conservation, refunds, and terminal states. The harness uses a test MAC to model authentication; it does not implement production signatures or persistent settlement.

A further check within the suite confirms the low-rank noise-recovery identity on 500 small integer examples. This tests algebra only, not cuPOW hardness, GPU kernels, or integer overflow at production dimensions. For a toy field $p = 101$, a fixed corrupted matrix escaped 95 of 10,000 random one-round checks (0.95%, near the $1/101$ bound). A negative control constructs a false result that passes when the challenge is known in advance.

Sampling detects a fraction of bad work



For a fixed corrupted fraction f and q independent uniform samples with replacement, detection is $1 - (1 - f)^q$, assuming each sampled bad unit is always detected. At $f = 1\%$, 16 samples detect only 14.85%; 300 detect 95.10%. At $f = 0.1\%$, even 600 samples detect only 45.14%. Sampling without replacement uses the corresponding hypergeometric probability.

Do not turn a sample into a full-proof badge

These probabilities require a fixed population, unpredictable sampling, available data, and a reliable check. Selective aborts, adaptive corruption, unbounded populations, colluding auditors, or an incorrect unit definition change the result. A small audit cannot establish complete LLM correctness.

14 / VALIDATION PLAN

The GPU benchmark that decides value

Proposed acceptance gates below are engineering targets. None has been demonstrated by the CPU reference experiments.

Compare at the same load	Purpose
A. Best approved ordinary serving configuration	Measures the real opportunity cost; may use a different approved precision profile.
B. Fixed INT7 profile with plain GEMM	Isolates profile quality and latency before mining.
C. Existing Pearl shape-gated miner	Establishes the actual baseline this proposal must improve.
D. Nacre GPU preparation; original selector	Isolates the new GPU kernel from scheduling changes.
E. Nacre GPU plus adaptive selection	Measures the combined mining and service effect.

Hold requests, token caps, caches, power limits, runtime revisions, and arrival traces fixed. Include prompt-length and concurrency sweeps, bursts, cancellations, and reloads. Measure prefill/decode separately, retain failures, repeat runs, and publish percentile uncertainty.

Proposed go/no-go gate	Measurement
Numerical correctness	No unexplained mismatch in allowed integer operations; no unapproved profile mutation or accepted invalid proof.
Service performance	At agreed loads, P95 first-token and P99 inter-token latency $\leq 1.05\times$ baseline B, with at least 95% of B's SLO-qualified delivered-token goodput.
Quality	Pre-register task-specific non-inferiority margins against A. Example starting targets: ≤ 0.5 percentage-point accuracy loss and $\leq 1\%$ relative perplexity increase, with sufficient sample power.
Business/energy value	Report cost and joules per delivered token, including quantization, proof, audit, transfer, cooling allocation, and stale work. Show totals before and after realized mining revenue.
Operational resilience	Bounded memory under queue saturation; durable replay prevention; safe template changes, node disconnects, proof failures, and restarts.

Report failures alongside latency, so dropped requests cannot improve the score. Label audit coverage and verification class; only fully proven execution belongs in a V2 token numerator.

15 / IMPLEMENTATION MAP

Where the prototype fits

Existing component	Proposed change
New nacre_gpu preparation adapter [15-17]	Bind canonical factors, compare AP/BP/U/V, preserve scale conversion, and dispatch to the measured faster path.
vllm_miner/config.py and config.yaml [5]	Keep shape gate. Add versioned service-budget, memory, proof-queue, and calibration settings.
vllm_miner/vllm_kernels.py [4]	Place the adaptive decision inside <code>_apply_weights_mining</code> , after fixed quantization and before noisy/plain GEMM selection.
New miner-side budget controller	Collect shape/device/profile cost statistics; plan permits; update a shared serving-budget ledger; export reasons for rejection.
miner_base/async_loop_manager.py [9]	Bound pending events, retained witness data, and proof submissions; define overflow behavior and reservation release.
Pearl gateway / node interface [3]	Consume current template/certificate rules; preserve existing candidate submission and verification.
New off-chain receipt service	Validate signed manifests, meter segments, manage idempotent funded settlement, and attach scoped audit reports.

Start on documented hardware

The inspected miner README documents sm90 H100/H200 support for its GPU tests. A broader capability check in the kernel is not evidence of end-to-end validation on every newer GPU. The initial correctness baseline should use that documented environment. [3,4]

For an RTX 5090, separately validate compiled kernels, numerical behavior, model/KV/proof memory coexistence, and serving cost. A general CUDA test does not establish full Pearl support. Windows is also a separate integration task; begin the full-stack comparison in the project's documented environment.

Compatibility boundary

v0.2 proposes miner and service software changes. It adds no block field, new difficulty function, proof datatype, on-chain model registry, issuance rule, or model whitelist to consensus. Every generated certificate must still pass the unmodified applicable Pearl node rules.

16 / DELIVERY PLAN

A staged path to a pilot

Illustrative engineering sequence for a small team with GPU access; timing is an estimate, not a delivery commitment.

Stage	Work product	Exit decision
Weeks 1-2 Baseline	Compile/test CUDA; pin Pearl and vLLM; compare canonical factors and certificates; collect baseline traces.	Stop if the numerical profile or hardware is unsuitable.
Weeks 3-4 Adaptive miner	Benchmark GPU preparation; add dispatch, resource caps, fallback, and shadow-mode service estimates.	Enable mining only on calibrated shapes after injected-failure tests.
Weeks 5-6 Receipts and payments	Implement signed manifests, atomic segment ledger, prepaid limits, and explicit verifier classes.	Pass restart, replay, duplicate-transfer, refund, and dispute scenarios.
Weeks 7-8 Independent evaluation	Run blinded workload comparisons, adversarial audits, privacy review, and proof-pipeline stress.	Accept only if the pre-registered service and quality gates hold.
Weeks 9-12 Limited pilot	Use bounded customer traffic, documented verification limits, operator rollback, and external review.	Expand only after measured benefit persists under real demand.

People and artifacts required

The core roles are an inference/CUDA engineer, a distributed-systems engineer, and a cryptography reviewer; one person may cover several roles in a research prototype. Deliver a reproducible container, pinned model manifest, baseline traces, load generator, receipt schema, failure-injection suite, and a measured cost report. A deployment budget should follow the baseline hardware measurements.

A useful first experiment

First compare the new GPU preparation against the existing noising kernels with identical canonical factors. Then replay one dense-model serving workload with fixed INT7 arithmetic, separately measuring the GPU kernel and scheduler changes. Require a net end-to-end gain before building the receipt service.

17 / RESEARCH EXTENSION

Toward a real proof of inference

A later version could replace operator-attested job binding with a proof of a precisely specified model computation. The relation would need to establish the following, for public job/output commitments and private model/input/trace witnesses:

```
R(job, output_commitment, work_reference; witness) =
  model_and_input_match_committed_manifest
  AND every_supported_model_transition_is_valid
  AND cache_and_sampling_transitions_are_valid
  AND output_matches_output_commitment
  AND referenced_mining_operands_equal_trace_operands
  AND referenced_Pearl_certificate_is_valid
```

The last two conditions are different. Adding a job ID to a hash or placing a receipt root in a coinbase commitment does not prove that the mined matrices came from that job. The proof needs an authenticated equality link between the actual execution trace and the operands committed by the work proof.

Research track	Evidence required before adoption
Full model verification	A canonical arithmetic specification covering nonlinear operators, attention, cache updates, routing where applicable, and output selection. Measure prover memory and time per real request.
FP8 / FP4 integration	A noise/encoding construction with a clear adversarial model, shortcut analysis, quantified approximation error, and model-quality validation.
Private inference	Define who must not see inputs or weights. TEEs, multi-party protocols, and full cryptographic approaches have different trust and performance assumptions.
Training or fine-tuning	Bind dataset commitments, optimizer state, update rules, and output checkpoints; study numerical stability and full-step verification costs.

Pearl's Polymath agenda explicitly treats efficient full inference proofs and lower-overhead encodings as research problems. The original arbitrary-matmul work also states a security conjecture; asymptotic overhead results do not establish zero measured overhead on today's serving stack. Nacre makes neither assumption disappear. [2,14]

18 / REVIEW CHECKLIST

Decision record and stop conditions

Decision	Reason
Build on cuPOW	A compatible miner experiment can be evaluated without inventing an unaudited replacement consensus primitive.
Prioritize dense inference	It narrows numerical, serving, and certificate-eligibility questions for the first experiment.
Protect service budgets	Mining yield is useful only if the combined service remains competitive with approved ordinary inference.
Keep rewards evidence-specific	A valid work proof, a model-execution proof, and independent customer demand are distinct claims.
Require a best-serving baseline	INT7 mining may lose to a faster or better-quality ordinary configuration even if it beats another mining mode.

Stop or redesign when any of these occurs

Quality falls outside the agreed margin. Move to another approved profile or model; do not hide a model change behind a mining toggle. **Proof and audit cost consumes the benefit.** Reduce the verification scope transparently or stop that service class. **Tail latency becomes unpredictable.** Disable mining for the affected workload until a new calibration passes.

Most demand is self-generated. Report it as declared usage, not independently verified economic demand. **Only one broker or model can participate profitably.** Revisit routing and pricing assumptions before promising a decentralized inference market. **The dominant cost is memory movement.** Extra GEMM mining may not address the actual bottleneck.

What this report establishes

A concrete GPU algorithm and CUDA prototype, explicit trust boundaries, an integration map, and reproducible CPU mechanism tests. It does not establish production speedup, mining profitability, cryptographic novelty, full-model proof efficiency, or a completed security audit.

Recommended next deliverable: a compiled, integrated GPU prototype and measured baseline comparison. Its result should determine whether Pearl should adopt, revise, or reject the scheduling proposal.

APPENDIX A / METHODS

Reproduce the reference experiments

The PDF embeds the CUDA prototype, its CPU math tests/results, and the scheduling reference/tests. The accompanying ZIP provides the same source files and build instructions. Extract them with a PDF reader that exposes attachments. The scheduling harness uses the Python standard library; the GPU-math oracle uses NumPy. No test submits mining work or payments.

```
python3 nacre_reference.py --out reproduced_results.json
```

Synthetic trace specification

Python random.Random seed 20261007; 10,000 independent batches; baseline batch time uniform in [8, 40] ms; deadline slack uniform in [1%, 15%] of baseline time. Each batch contains 12 candidate operations, each shape-eligible with probability 0.75. An exogenous pressure gate disables Nacre in 12% of batches.

Each candidate has a nominal upper latency reserve uniform in [0.04, 0.24] ms and actual extra time equal to that reserve times a uniform [0.60, 1.00] factor. Independently, 1% of candidates receive a 2x shock. Synthetic work weights are uniform in [1, 20]. Nacre uses a 0.25 ms guard and greedy work/reserve selection; budget settings are 2%, 5%, and 10%.

The sensitivity run scales reservations by 1.0, 0.75, or 0.5 on the same trace. Percentiles use the nearest-rank method. The model does not simulate arrivals, token lengths, actual proof queues, thermal feedback, stale-work rates, or price/difficulty changes. It assumes serial additive costs and a known baseline. No inference service guarantee follows from it.

CPU accounting and audit scope

The ledger is an in-memory single-broker model. It checks a reduced schema, exact integer budgets, contiguous output ranges, and terminal states. Authentication is modeled with a fixed test HMAC; production signing, authorization, concurrency, persistence, chain reorganization, and fee-rate validation are outside this harness. Algebraic audits use Python integers, not CUDA or a production proof circuit.

Reference-source SHA-256

```
b20bcb3b72a89a5783c9d46bdad85a24  
9e2101c872f8d3069d97e28fc1a6d638
```

The original Pearl repository was inspected read-only. This report contains proposed integration steps; no upstream modification, live deployment, or GPU benchmark was performed.

APPENDIX B / PRIMARY SOURCES

Sources and audit trail

All online sources accessed 7 October 2026. Bracketed numbers in the report refer to these entries.

Repository references are pinned to 2f8b770cac8f8b74be05c1fc51c2baeb76ce0701. The checkout records a 4 October 2026 commit. Repository descriptions are observations of that snapshot; they do not attest to live network state.

[1] Pearl Research Labs. INT whitepaper.

Protocol architecture, integer work construction, UTXO network and block timing.

pearlresearch.ai/research/int-whitepaper

[2] Pearl Polymath Project.

Official open questions on lower-overhead encodings, quantization, training, ASIC economics, and full proof-of-inference.

pearlpolymath.com/

[3] Pearl repository: miner/README.md.

Current serving integration and documented GPU-test support.

Pinned source: [miner/README.md](#)

[4] Pearl repository: vllm_kernels.py.

Mining/non-mining numerical modes, noisy/plain dispatch, and hardware capability checks.

Pinned source: [miner/vllm-miner/src/vllm_miner/vllm_kernels.py](#)

[5] Pearl repository: config.py and config.yaml.

Shape eligibility logic and checked-in threshold defaults.

Pinned source: [miner/vllm-miner/src/vllm_miner/config.py](#)

Companion: [config.yaml](#)

[6] Pearl repository: zk-pow/src/api/proof.rs.

Proof types, mining configuration, operand commitments, and matrix datatype.

Pinned source: [zk-pow/src/api/proof.rs](#)

[7] Pearl repository: node/chaincfg/params.go.

Network parameters and activation heights; inspect target network, not just one hard-coded profile.

Pinned source: [node/chaincfg/params.go](#)

APPENDIX B / PRIMARY SOURCES

Sources and audit trail, continued

[8] Pearl repository: [node/blockchain/validate.go](#) and [seed.rs](#).

Certificate eligibility and version-dependent salted-seed behavior.

Pinned source: [node/blockchain/validate.go](#)

Companion: [zk-pow/src/api/seed.rs](#)

[9] Pearl repository: [async_loop_manager.py](#).

Existing asynchronous CUDA events and proof-submission pathway.

Pinned source: [miner/miner-base/src/miner_base/async_loop_manager.py](#)

[10] vLLM. Optimization and Tuning.

Serving-scheduler context, chunked prefill, decode priority, and memory pressure.

[docs.vllm.ai/en/latest/configuration/optimization/](#)

[11] Together AI. Partnership announcement, 15 May 2026.

Primary announcement of a Pearl-enabled inference endpoint; promotional performance/economic claims not independently reproduced here.

[www.together.ai/blog/together-ai-partners-with-pearl-research-labs](#)

[12] Tramer and Boneh. Slalom. ICLR 2019.

Verifiable delegated neural-network linear operations with trusted hardware; relevant background, not a generic trustless LLM proof.

[arxiv.org/abs/1806.03287](#)

[13] Sun, Li, and Zhang. zkLLM. ACM CCS 2024.

Specialized zero-knowledge proof techniques for transformer computations and non-arithmetic operations.

[arxiv.org/abs/2404.16109](#)

[14] Komargodski and Weinstein. Proofs of Useful Work from Arbitrary Matrix Multiplication. v4, 13 November 2025.

Foundational construction and stated security conjecture.

[arxiv.org/abs/2504.09971v4](#)

Evidence convention: sourced statements describe existing work. Nacre policy choices, proposed targets, equations, failure analyses, and reference experiments are this report's design and analysis. No affiliation with or endorsement by Pearl Research Labs is implied.

APPENDIX B / PRIMARY SOURCES

GPU kernel sources and build context

[15] Pearl: noise_generation_kernel.h.

Structured two-index sparse factors and current dense noise range.

Pinned source: [miner/pearl-gemm/csrc/gemm/noise_generation_kernel.h](#)

[16] Pearl: pearl_noisingA_kernel.h and pearl_noisingB_kernel.h.

Existing WGMMA preparation; the proposed sparse row kernel is an alternative GPU mapping.

Pinned source: [miner/pearl-gemm/csrc/gemm/pearl_noisingA_kernel.h](#)

[17] Pearl: collective_epilogue.hpp and pearl_gemm_constants.hpp.

Existing correction, scale constants, and floating-point epilogue that integration must preserve.

Pinned source: [miner/pearl-gemm/csrc/gemm/collective_epilogue.hpp](#)

[18] NVIDIA. CUDA C++ Language Extensions.

Shared-memory integer atomics and block synchronization. No GPU performance claim is inferred from the programming guide.

docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/cpp-language-extensions.html

[19] NVIDIA. CUDA Compiler Driver.

Compiler options including native architecture targeting.

docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/index.html

Companion: [pearl_noisingB_kernel.h](#)

Companion: [pearl_gemm_constants.hpp](#)

Authorship and status: the Nacre GPU source is an independent prototype implementing the transformation described here. It reuses Pearl's published mathematical structure conceptually; it does not include or replace Pearl's main mining kernel, cryptographic noise generator, or proof verifier. Original prototype source is supplied under the MIT license in the source package.